

Propositions as Types

Philip Wadler

University of Edinburgh

Lambda Days

Kraków, Thursday 18 February 2016

Part I

Computability

Euclid (325–265 BCE) / Al Khwarizmi (780–850 CE)



Algorithm, formalised



Alonzo Church: Lambda calculus

An unsolvable problem of elementary number theory, *Bulletin the American Mathematical Society*, May 1935



Kurt Gödel: Recursive functions

Stephen Kleene, General recursive functions of natural numbers, *Bulletin the American Mathematical Society*, July 1935



Alan M. Turing: Turing machines

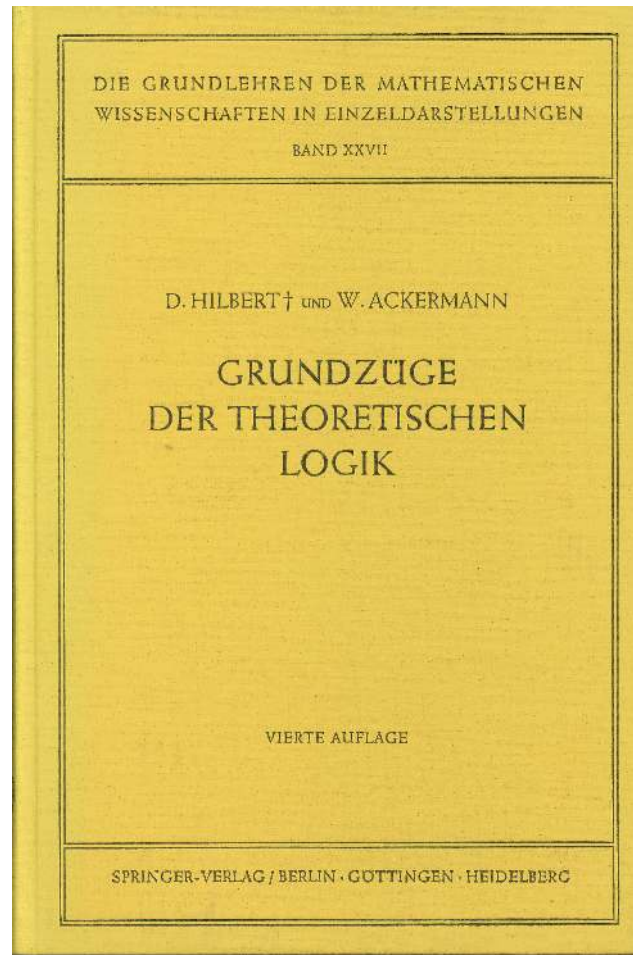
On computable numbers, with an application to the *Entscheidungsproblem*, *Proceedings of the London Mathematical Society*, received 25 May 1936



David Hilbert (1862–1943)



David Hilbert (1928) — Entscheidungsproblem



Kurt Gödel (1906–1978)



Kurt Gödel (1930) — Incompleteness

ON FORMALLY UNDECIDABLE PROPOSITIONS OF *PRINCIPIA* *MATHEMATICA* AND RELATED SYSTEMS I¹ (1931)

42. $Ax(x) \equiv Z \cdot Ax(x) \vee A \cdot Ax(x) \vee L_1 \cdot Ax(x) \vee L_2 \cdot Ax(x) \vee R \cdot Ax(x) \vee M \cdot Ax(x)$,
 x is an AXIOM.

43. $Fl(x, y, z) \equiv y = z \text{ Imp } x \vee (Ev)[v \leq x \ \& \ \text{Var}(v) \ \& \ x = v \text{ Gen } y]$,
 x is an IMMEDIATE CONSEQUENCE of y and z .

44. $Bw(x) \equiv (n)\{0 < n \leq l(x) \rightarrow Ax(n \text{ Gl } x) \vee (Ep, q)[0 < p, q < n \ \& \ Fl(n \text{ Gl } x, p \text{ Gl } x, q \text{ Gl } x)] \} \ \& \ l(x) > 0$,
 x is a PROOF ARRAY (a finite sequence of FORMULAS, each of which is either an AXIOM or an IMMEDIATE CONSEQUENCE of two of the preceding FORMULAS).

45. $x B y \equiv Bw(x) \ \& \ [l(x)] \text{ Gl } x = y$,
 x is a PROOF of the FORMULA y .

46. $Bew(x) \equiv (Ey)y B x$,
 x is a PROVABLE FORMULA. ($Bew(x)$ is the only one of the notions 1–46 of which we cannot assert that it is recursive.)

“This statement is not provable”



Alonzo Church (1903–1995)



Alonzo Church (1936) — Lambda Calculus

AN UNSOLVABLE PROBLEM OF ELEMENTARY NUMBER THEORY.¹

By ALONZO CHURCH.

The purpose of the present paper is to propose a definition of effective calculability⁸ which is thought to correspond satisfactorily to the somewhat vague intuitive notion in terms of which problems of this class are often stated, and to show, by means of an example, that not every problem of this class is solvable.

...

We introduce at once the following infinite list of abbreviations,

$$\begin{aligned} 1 &\rightarrow \lambda ab \cdot a(b), \\ 2 &\rightarrow \lambda ab \cdot a(a(b)), \\ 3 &\rightarrow \lambda ab \cdot a(a(a(b))), \end{aligned}$$

and so on, each positive integer in Arabic notation standing for a formula of the form $\lambda ab \cdot a(a(\cdots a(b) \cdots))$.

Alonzo Church (1932) — λ -calculus

$$\begin{array}{lcl} L, M, N & ::= & x \\ & | & (\lambda x. N) \\ & | & (L M) \end{array}$$



Kurt Gödel (1906–1978)



Kurt Gödel (1936) — Recursive Functions

General recursive functions of natural numbers¹⁾.

Von

S. C. Kleene in Madison (Wis., U.S.A.).

The substitution

$$1) \quad \varphi(x_1, \dots, x_n) = \theta(\chi_1(x_1, \dots, x_n), \dots, \chi_m(x_1, \dots, x_n)),$$

and the ordinary recursion with respect to one variable

$$(2) \quad \begin{aligned} \varphi(0, x_2, \dots, x_n) &= \psi(x_2, \dots, x_n) \\ \varphi(y+1, x_2, \dots, x_n) &= \chi(y, \varphi(y, x_2, \dots, x_n), x_2, \dots, x_n), \end{aligned}$$

where $\theta, \chi_1, \dots, \chi_m, \psi, \chi$ are given functions of natural numbers, are examples of the definition of a function φ by equations which provide a step by step process for computing the value $\varphi(k_1, \dots, k_n)$ for any given set $k_1, \dots, k_n$ of natural numbers. It is known that there are other definitions of this sort, e. g. certain recursions with respect to two or more variables simultaneously, which cannot be reduced to a succession of substitutions and ordinary recursions²⁾. Hence, a characterization of the notion of recursive definition in general, which would include all these cases, is desirable. A definition of general recursive function of natural numbers was suggested by Herbrand to Gödel, and was used by Gödel with an important modification in a series of lectures at Princeton in 1934. In this paper we offer several observations on general recursive functions, using essentially Gödel's form of the definition.

Alan Turing (1912–1954)



Alan Turing (1936) — Turing Machine

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

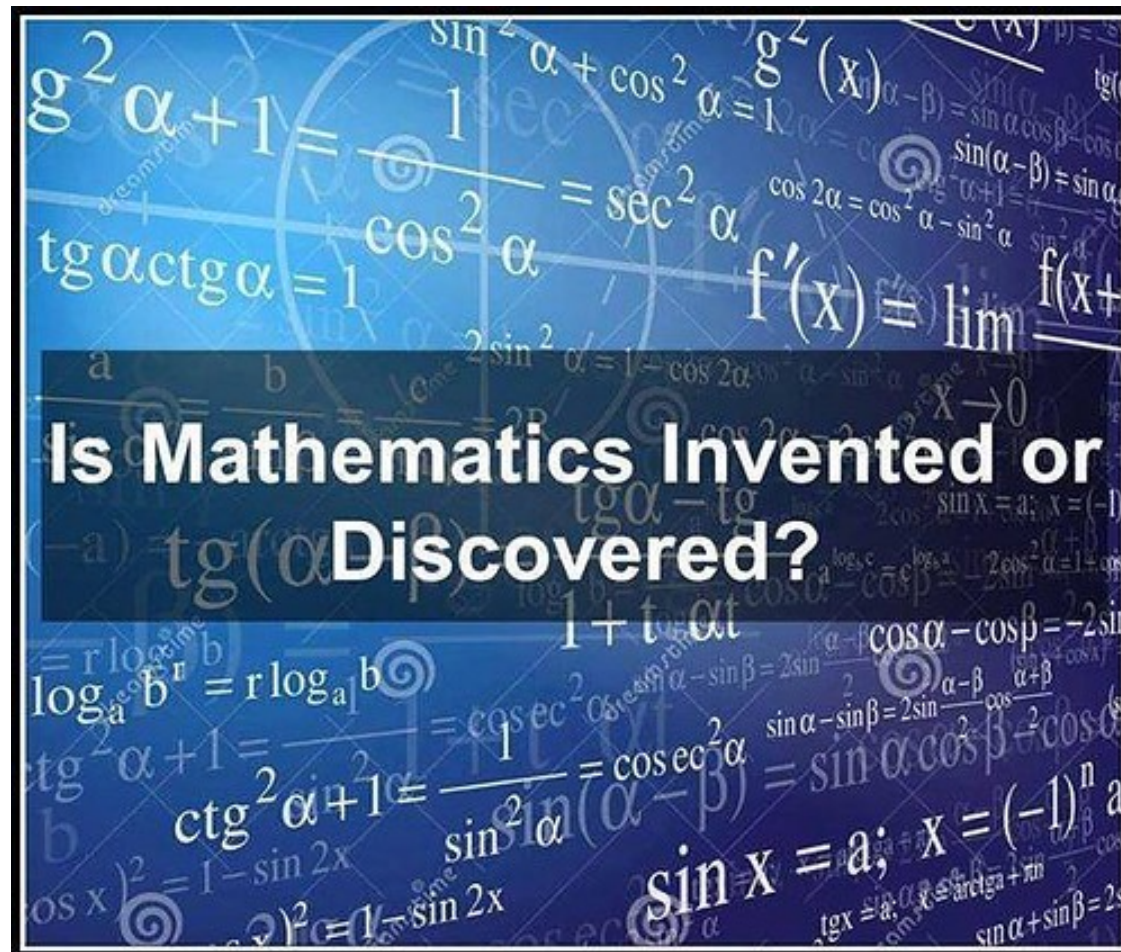
By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

The “computable” numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means.

...

In §§ 9, 10 I give some arguments with the intention of showing that the computable numbers include all numbers which could naturally be regarded as computable. In particular, I show that certain large classes of numbers are computable. They include, for instance, the real parts of all algebraic numbers, the real parts of the zeros of the Bessel functions, the numbers π , e , etc. The computable numbers do not, however, include all definable numbers, and an example is given of a definable number which is not computable.





Kurt Gödel, 28



David Hilbert, 68



Alan Turing, 23



Alonzo Church, 33



Kurt Gödel, 30

Part II

Propositions as Types

Gerhard Gentzen (1909–1945)



Gerhard Gentzen (1935) — Natural Deduction

$\&-I$ $\frac{\mathcal{A} \quad \mathcal{B}}{\mathcal{A} \& \mathcal{B}}$	$\&-E$ $\frac{\mathcal{A} \& \mathcal{B}}{\mathcal{A}} \quad \frac{\mathcal{A} \& \mathcal{B}}{\mathcal{B}}$	$\vee-I$ $\frac{\mathcal{A}}{\mathcal{A} \vee \mathcal{B}} \quad \frac{\mathcal{B}}{\mathcal{A} \vee \mathcal{B}}$	$\vee-E$ $\frac{\mathcal{A} \vee \mathcal{B} \quad \begin{array}{c} [\mathcal{A}] \\ \vdots \\ \mathcal{C} \end{array} \quad \begin{array}{c} [\mathcal{B}] \\ \vdots \\ \mathcal{C} \end{array}}{\mathcal{C}}$
$\forall-I$ $\frac{\mathcal{F}a}{\forall x \mathcal{F}x}$	$\forall-E$ $\frac{\forall x \mathcal{F}x}{\mathcal{F}a}$	$\exists-I$ $\frac{\mathcal{F}a}{\exists x \mathcal{F}x}$	$\exists-E$ $\frac{\exists x \mathcal{F}x \quad \begin{array}{c} [\mathcal{F}a] \\ \vdots \\ \mathcal{C} \end{array}}{\mathcal{C}}$
$\supset-I$ $\frac{\begin{array}{c} [\mathcal{A}] \\ \vdots \\ \mathcal{B} \end{array}}{\mathcal{A} \supset \mathcal{B}}$	$\supset-E$ $\frac{\mathcal{A} \quad \mathcal{A} \supset \mathcal{B}}{\mathcal{B}}$	$\neg-I$ $\frac{\begin{array}{c} [\mathcal{A}] \\ \vdots \\ \mathcal{A} \end{array}}{\neg \mathcal{A}}$	$\neg-E$ $\frac{\mathcal{A} \quad \neg \mathcal{A}}{\mathcal{A}} \quad \frac{\mathcal{A}}{\mathcal{D}} .$

Gerhard Gentzen (1935) — Natural Deduction

$$\frac{\begin{array}{c} [A]^x \\ \vdots \\ B \end{array}}{A \supset B} \supset\text{-I}^x \qquad \frac{A \supset B \quad A}{B} \supset\text{-E}$$

$$\frac{A \quad B}{A \& B} \&\text{-I} \qquad \frac{A \& B}{A} \&\text{-E}_0 \qquad \frac{A \& B}{B} \&\text{-E}_1$$

A proof

$$\frac{\frac{[B \& A]^z}{A} \&\text{-E}_1 \quad \frac{[B \& A]^z}{B} \&\text{-E}_0}{\frac{A \& B}{(B \& A) \supset (A \& B)} \supset\text{-I}^z} \&\text{-I}$$

Simplifying proofs

$$\begin{array}{c}
 [A]^x \\
 \vdots \\
 B \\
 \hline
 A \supset B \quad \supset\text{-I}^x
 \end{array}
 \quad
 \begin{array}{c}
 \vdots \\
 A
 \end{array}
 \quad
 \supset\text{-E}
 \Rightarrow
 \begin{array}{c}
 \vdots \\
 A \\
 \vdots \\
 B
 \end{array}$$

$$\begin{array}{c}
 \vdots \quad \vdots \\
 A \quad B \\
 \hline
 A \& B \quad \&\text{-I} \\
 \hline
 A \quad \&\text{-E}_0
 \end{array}
 \Rightarrow
 \begin{array}{c}
 \vdots \\
 A
 \end{array}$$

Simplifying a proof

$$\begin{array}{c}
 \frac{[B \& A]^z}{A} \&-E_1 \quad \frac{[B \& A]^z}{B} \&-E_0 \\
 \hline
 A \& B \quad \&-I \\
 \hline
 (B \& A) \supset (A \& B) \quad \supset-I^z \\
 \hline
 A \& B
 \end{array}
 \quad
 \begin{array}{c}
 \frac{[B]^y \quad [A]^x}{B \& A} \&-I \\
 \hline
 B \& A \quad \supset-E
 \end{array}$$

Simplifying a proof

$$\begin{array}{c}
 \frac{[B \& A]^z}{A} \&-E_1 \quad \frac{[B \& A]^z}{B} \&-E_0 \\
 \hline
 A \& B \quad \&-I \\
 \hline
 (B \& A) \supset (A \& B) \supset-I^z \\
 \hline
 \frac{(B \& A) \supset (A \& B) \quad \frac{[B]^y \quad [A]^x}{B \& A} \&-I}{A \& B} \supset-E
 \end{array}$$

$\Downarrow$

$$\begin{array}{c}
 \frac{[B]^y \quad [A]^x}{B \& A} \&-I \quad \frac{[B]^y \quad [A]^x}{B \& A} \&-I \\
 \frac{\frac{B \& A}{A} \&-E_1 \quad \frac{B \& A}{B} \&-E_0}{A \& B} \&-I
 \end{array}$$

Simplifying a proof

$$\begin{array}{c}
 \frac{[B \& A]^z}{A} \&-E_1 \quad \frac{[B \& A]^z}{B} \&-E_0 \\
 \hline
 A \& B \quad \&-I \\
 \hline
 (B \& A) \supset (A \& B) \supset-I^z \\
 \hline
 \frac{(B \& A) \supset (A \& B) \quad \frac{[B]^y \quad [A]^x}{B \& A} \&-I}{A \& B} \supset-E
 \end{array}$$

$\Downarrow$

$$\begin{array}{c}
 \frac{[B]^y \quad [A]^x}{B \& A} \&-I \quad \frac{[B]^y \quad [A]^x}{B \& A} \&-I \\
 \hline
 \frac{B \& A}{A} \&-E_1 \quad \frac{B \& A}{B} \&-E_0 \\
 \hline
 A \& B \quad \&-I
 \end{array}$$

$\Downarrow$

$$\frac{[A]^x \quad [B]^y}{A \& B} \&-I$$

Alonzo Church (1903–1995)



Alonzo Church (1940) — Typed λ -calculus

$$\frac{\begin{array}{c} [x : A]^x \\ \vdots \\ N : B \end{array}}{\lambda x. N : A \supset B} \supset\text{-I}^x \qquad \frac{L : A \supset B \quad M : A}{LM : B} \supset\text{-E}$$

$$\frac{M : A \quad N : B}{(M, N) : A \& B} \&\text{-I} \qquad \frac{L : A \& B}{\text{fst } L : A} \&\text{-E}_0 \qquad \frac{L : A \& B}{\text{snd } L : B} \&\text{-E}_1$$

A program

$$\frac{\frac{[z : B \& A]^z}{\text{snd } z : A} \&\text{-E}_1 \quad \frac{[z : B \& A]^z}{\text{fst } z : B} \&\text{-E}_0}{\frac{}{(\text{snd } z, \text{fst } z) : A \& B} \&\text{-I}} \supset\text{-I}^z$$
$$\lambda z. (\text{snd } z, \text{fst } z) : (B \& A) \supset (A \& B)$$

Evaluating programs

$$\frac{
 \frac{
 \begin{array}{c} [x : A]^x \\ \vdots \\ N : B \end{array}
 }{
 \lambda x. N : A \supset B
 } \supset\text{-I}^x
 \quad
 \begin{array}{c} \vdots \\ M : A \end{array}
 }{
 (\lambda x. N) M : B
 } \supset\text{-E}
 \Rightarrow
 \begin{array}{c} \vdots \\ M : A \\ \vdots \\ N\{M/x\} : B \end{array}$$

$$\frac{
 \frac{
 \begin{array}{c} \vdots \\ M : A \end{array}
 \quad
 \begin{array}{c} \vdots \\ N : B \end{array}
 }{
 (M, N) : A \& B
 } \&\text{-I}
 }{
 \text{fst} (M, N) : A
 } \&\text{-E}_0
 \Rightarrow
 \begin{array}{c} \vdots \\ M : A \end{array}$$

Alan Turing (1942)

AN EARLY PROOF OF NORMALIZATION
BY A.M. TURING

R.O. Gandy

*Mathematical Institute, 24-29 St. Giles,
Oxford OX1 3LB, UK*

Dedicated to H.B. Curry on the occasion of his 80th birthday

In the extract printed below, Turing shows that every formula of Church's simple type theory has a normal form. The extract is the first page of an unpublished (and incomplete) typescript entitled 'Some theorems about Church's system'. (Turing left his manuscripts to me; they are deposited in the library of King's College, Cambridge). An account of this system was published by Church in 'A formulation of the simple theory of types' (J. Symbolic Logic 5 (1940), pp. 56-68). Church had previously shown that the system is consistent.

Evaluating a program

$$\begin{array}{c}
 \frac{[z : B \& A]^z}{\text{snd } z : A} \&-E_1 \quad \frac{[z : B \& A]^z}{\text{fst } z : B} \&-E_0 \\
 \hline
 \frac{(\text{snd } z, \text{fst } z) : A \& B}{\lambda z. (\text{snd } z, \text{fst } z) : (B \& A) \supset (A \& B)} \&-I \\
 \hline
 \frac{\lambda z. (\text{snd } z, \text{fst } z) : (B \& A) \supset (A \& B) \quad \frac{[y : B]^y \quad [x : A]^x}{(y, x) : B \& A} \&-I}{(\lambda z. (\text{snd } z, \text{fst } z)) (y, x) : A \& B} \supset-E
 \end{array}$$

Evaluating a program

$$\begin{array}{c}
 \frac{[z : B \& A]^z}{\text{snd } z : A} \&-E_1 \quad \frac{[z : B \& A]^z}{\text{fst } z : B} \&-E_0 \\
 \hline
 (\text{snd } z, \text{fst } z) : A \& B \quad \&-I \\
 \hline
 \lambda z. (\text{snd } z, \text{fst } z) : (B \& A) \supset (A \& B) \quad \supset-I^z \quad \frac{[y : B]^y \quad [x : A]^x}{(y, x) : B \& A} \&-I \\
 \hline
 (\lambda z. (\text{snd } z, \text{fst } z)) (y, x) : A \& B \quad \supset-E \\
 \hline
 (\lambda z. (\text{snd } z, \text{fst } z)) (y, x) : A \& B \\
 \Downarrow \\
 \frac{\frac{[y : B]^y \quad [x : A]^x}{(y, x) : B \& A} \&-I}{\text{snd } (y, x) : A} \&-E_1 \quad \frac{\frac{[y : B]^y \quad [x : A]^x}{(y, x) : B \& A} \&-I}{\text{fst } (y, x) : B} \&-E_0 \\
 \hline
 (\text{snd } (y, x), \text{fst } (y, x)) : A \& B \quad \&-I
 \end{array}$$

Evaluating a program

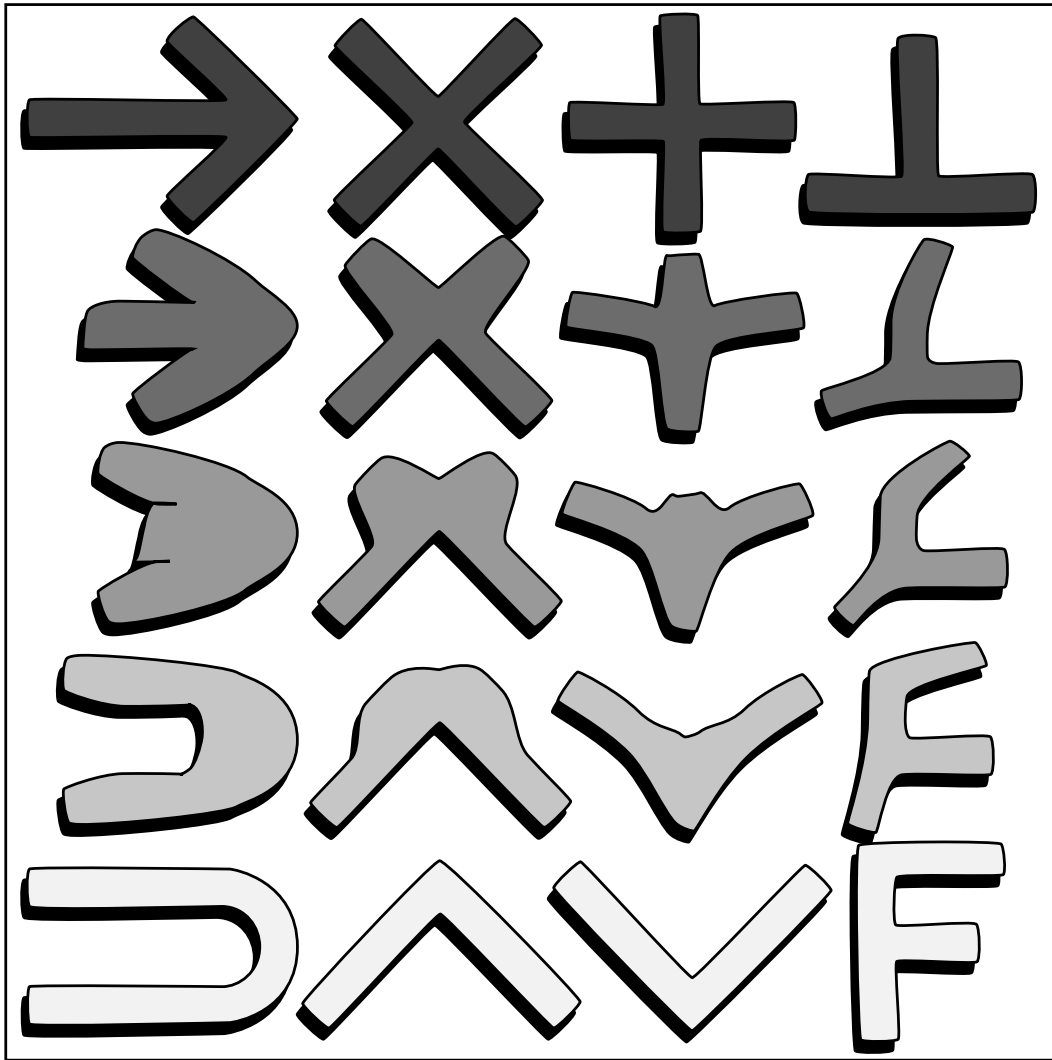
$$\begin{array}{c}
 \frac{[z : B \& A]^z}{\text{snd } z : A} \&-E_1 \quad \frac{[z : B \& A]^z}{\text{fst } z : B} \&-E_0 \\
 \hline
 (\text{snd } z, \text{fst } z) : A \& B \quad \&-I \\
 \hline
 \lambda z. (\text{snd } z, \text{fst } z) : (B \& A) \supset (A \& B) \quad \supset-I^z \quad \frac{[y : B]^y \quad [x : A]^x}{(y, x) : B \& A} \&-I \\
 \hline
 (\lambda z. (\text{snd } z, \text{fst } z)) (y, x) : A \& B \quad \supset-E
 \end{array}$$

$\Downarrow$

$$\begin{array}{c}
 \frac{[y : B]^y \quad [x : A]^x}{(y, x) : B \& A} \&-I \quad \frac{[y : B]^y \quad [x : A]^x}{(y, x) : B \& A} \&-I \\
 \hline
 \frac{\text{snd } (y, x) : A \quad \text{fst } (y, x) : B}{(\text{snd } (y, x), \text{fst } (y, x)) : A \& B} \&-I
 \end{array}$$

$\Downarrow$

$$\frac{[x : A]^x \quad [y : B]^y}{(x, y) : A \& B} \&-I$$



LC'90

The Curry-Howard homeomorphism

Haskell Curry (1900–1982) / William Howard (1926–)



Howard 1980

THE FORMULAE-AS-TYPES NOTION OF CONSTRUCTION

W. A. Howard

*Department of Mathematics, University of
Illinois at Chicago Circle, Chicago, Illinois 60680, U.S.A.*

Dedicated to H. B. Curry on the occasion of his 80th birthday.

The following consists of notes which were privately circulated in 1969. Since they have been referred to a few times in the literature, it seems worth while to publish them. They have been rearranged for easier reading, and some inessential corrections have been made.

Curry-Howard correspondence

propositions *as* types

proofs *as* programs

normalisation of proofs *as* evaluation of programs

Curry-Howard correspondence

Natural Deduction $\leftrightarrow$ Typed Lambda Calculus

Gentzen (1935) Church (1940)

Type Schemes $\leftrightarrow$ ML Type System

Hindley (1969) Milner (1975)

System F $\leftrightarrow$ Polymorphic Lambda Calculus

Girard (1972) Reynolds (1974)

Modal Logic $\leftrightarrow$ Monads (state, exceptions)

Lewis (1910) Kleisli (1965), Moggi (1987)

Classical-Intuitionistic Embedding $\leftrightarrow$ Continuation Passing Style

Gödel (1933) Reynolds (1972)

Linear Logic $\leftrightarrow$ Session Types

Girard (1987) Honda (1993)

Functional Languages

- **Lisp** (McCarthy, 1960)
- **Iswim** (Landin, 1966)
- **Scheme** (Steele and Sussman, 1975)
- **ML** (Milner, Gordon, Wadsworth, 1979)
- **Haskell** (Hudak, Hughes, Peyton Jones, and Wadler, 1987)
- **O'Caml** (Leroy, 1996)
- **Erlang** (Armstrong, Virding, Williams, 1996)
- **Scala** (Odersky, 2004)
- **F#** (Syme, 2006)
- **Clojure** (Hickey, 2007)
- **Elm** (Czaplicki, 2012)

Proof assistants

- [Automath](#) (de Bruijn, 1970)
- [Type Theory](#) (Martin Löf, 1975)
- [Mizar](#) (Trybulec, 1975)
- [ML/LCF](#) (Milner, Gordon, and Wadsworth, 1979)
- [NuPrl](#) (Constable, 1985)
- [HOL](#) (Gordon and Melham, 1988)
- [Coq](#) (Huet and Coquand, 1988)
- [Isabelle](#) (Paulson, 1993)
- [Epigram](#) (McBride and McKinna, 2004)
- [Agda](#) (Norell, 2005)

Propositions as Types

Philip Wadler

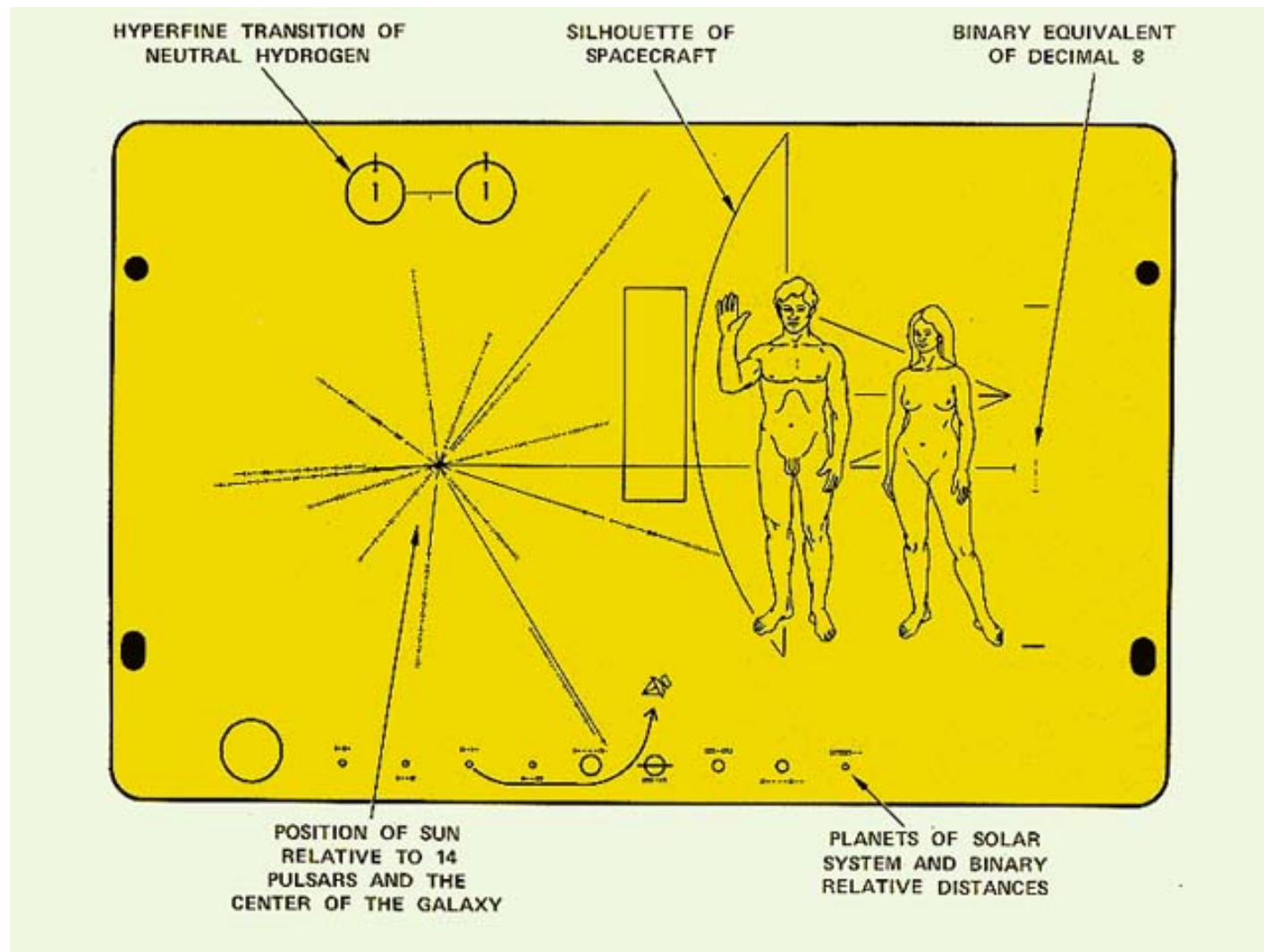
Communications of the ACM, 58(12):75–84

December 2015

Part III

Conclusion: Philosophy

Let's talk to aliens!



Independence Day



A universal programming language?



Multiverses



Lambda is Omniversal

